SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I T.Chaitra(192425217)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T.Chaitra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

Q1. Design an Autoencoder-Based Feature Learning System

1. Problem

A manufacturing company collects **high-dimensional sensor data** such as:

- Temperature
- Pressure
- Vibration
- Speed
- Voltage
- Current
- Sound
- Humidity

The objective is to learn a compact representation of the sensor data and use it to detect defective products.

An **Undercomplete Autoencoder** is suitable because it compresses high-dimensional input into a smaller latent representation.

2. Input and Output

Input: High-dimensional sensor vector

$$X = [x_1, x_2, .., .. x_n]$$

Output: Reconstructed sensor vector

$$\hat{X}$$

The difference between $X$and $\hat{X}$is measured using reconstruction loss.

3. Proposed Architecture

High-Dimensional Sensor Data

↓

Input Layer

n features

↓

Dense Layer – 128

ReLU

↓

Dense Layer – 64

ReLU

↓

LATENT SPACE

16 features

↓

Dense Layer – 64

ReLU

↓

Dense Layer – 128

ReLU

↓

Output Layer – n

Linear

↓

Reconstructed Data

Encoder

The encoder converts the original high-dimensional data into a smaller representation.

Input → 128 → 64 → 16

Latent Representation

The **16-dimensional latent vector** contains the most useful information.

For example:

Original data = 100 features

↓

Latent vector = 16 features

This is called an **undercomplete representation** because:

$$Latent\ Dimension < Input\ Dimension$$

Decoder

16 → 64 → 128 → 100

The decoder reconstructs the original sensor input.

4. Activation Functions

For hidden layers:

ReLU

$$ReLU(x) = max(0, x)$$

For the output layer, **Linear activation** can be used for continuous sensor values.

5. Reconstruction Loss

Use **Mean Squared Error (MSE)**:

$$L = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

The model tries to minimize the difference between the original and reconstructed sensor data.

6. How Dimensionality Reduction Happens

Suppose the company has:

100 sensor features

↓

Encoder

↓

16 latent features

↓

Decoder

↓

100 reconstructed features

The bottleneck forces the network to learn the **most important patterns** instead of simply copying the input.

7. Training

1. Normalize sensor data.
2. Feed sensor vector to encoder.
3. Generate latent representation.
4. Decode the latent representation.
5. Calculate reconstruction loss.
6. Use backpropagation to update weights.
7. Repeat until reconstruction error is minimized.

8. Defect Detection

After training, reconstruction error can be used for anomaly detection.

Sensor Data

↓

Autoencoder

↓

Reconstruction Error

↓

Compare with Threshold

↓

```
┌──────────────────────────┐
│ Low Error    │ → Normal  │
│ High Error   │ → Defective│
└──────────────────────────┘
```

Evaluation

Use:

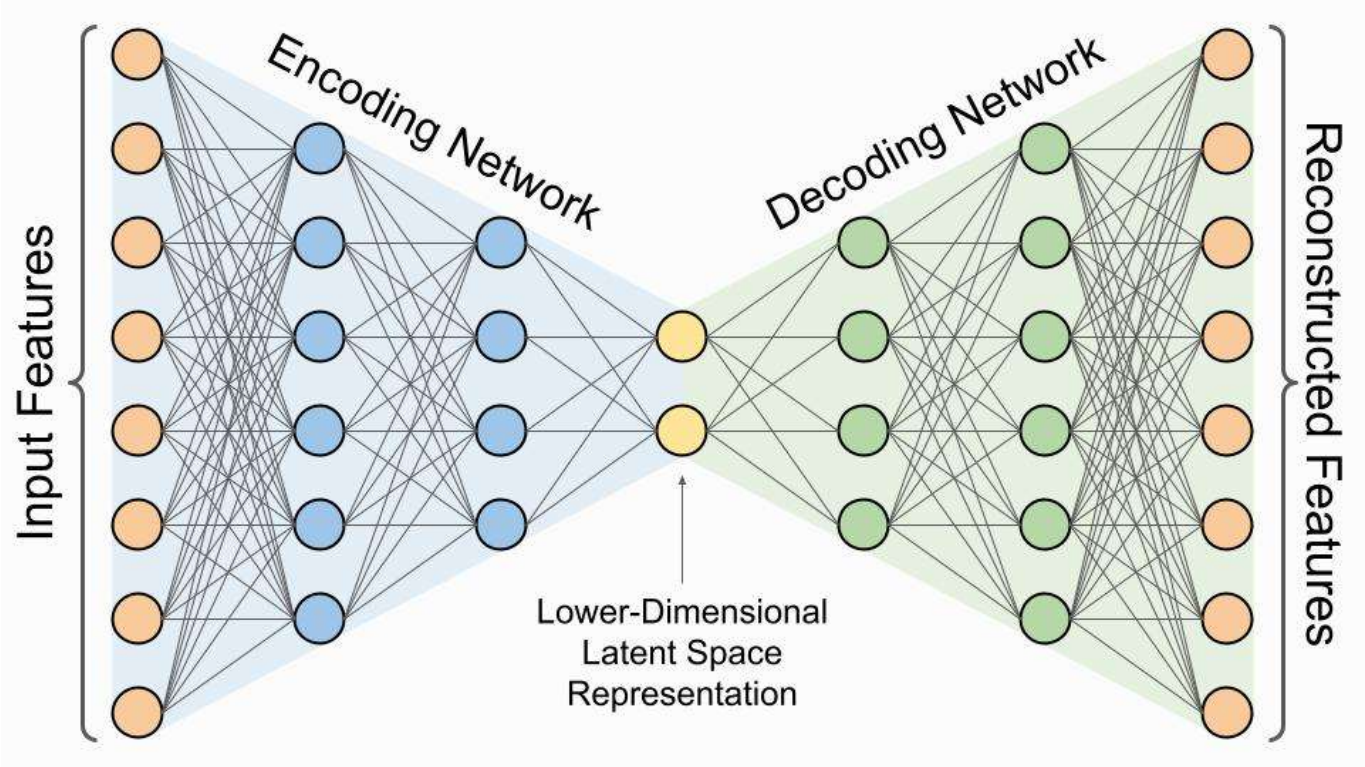
- Reconstruction MSE
- MAE

- Precision
- Recall
- F1-score
- ROC-AUC for defect detection

Conclusion

The undercomplete autoencoder reduces dimensionality by forcing high-dimensional sensor data through a small latent space. The learned latent representation captures important sensor patterns, while high reconstruction error can indicate defective products.

Memory:
Large Input → Small Latent Space → Reconstruction → Error → Defect



Q2. Design a Robust Regularized Autoencoder

1. Problem

Healthcare data may contain:

- Noise
- Missing values
- Irrelevant features
- Measurement errors

A normal autoencoder may learn unwanted patterns. Therefore, a **regularized autoencoder** is required.

I would use a **Denoising Autoencoder**, which is a suitable regularization strategy.

2. Input and Output

Input: Patient-record features

Example:

Age, BP, Glucose, Cholesterol,

Heart Rate, BMI, etc.

Output: Reconstructed clean patient representation.

3. Architecture

Patient Data

↓

Normalization

↓

Add Noise / Corruption

↓

Encoder

↓

128 → 64 → 16

↓

Latent Representation

↓

Decoder

↓

64 → 128

↓

Reconstructed Clean Data

↓

Downstream Classifier

↓

Disease Class

Encoder

Input

↓

Dense(128, ReLU)

↓

Dense(64, ReLU)

↓

Dense(16, ReLU)

Decoder

16

↓

64, ReLU

↓

128, ReLU

↓

Output

4. Regularization Mechanism

The input is deliberately corrupted by adding noise.

For example:

Original:

[80, 120, 95, 70]

Noisy:

[80, 0, 95, 70]

The model receives the noisy input but tries to reconstruct the original clean input.

This forces the model to learn **robust features**.

5. Training Objective

The model minimizes reconstruction loss:

$$L_{reconstruction} = \frac{1}{n}\sum(x - \hat{x})^2$$

A regularization term can also be included:

$$L = L_{reconstruction} + \lambda L_{regularization}$$

where λcontrols the strength of regularization.

6. How It Reduces Overfitting

Without regularization:

Input → Encoder → Decoder → Copy input

The model may simply memorize training data.

With noise:

Noisy Input → Encoder → Latent → Decoder → Clean Input

The model must learn important underlying patterns.

Benefits

- Robust to noise
- Better generalization
- Reduced overfitting
- Useful feature extraction
- Better downstream classification

Evaluation

Use:

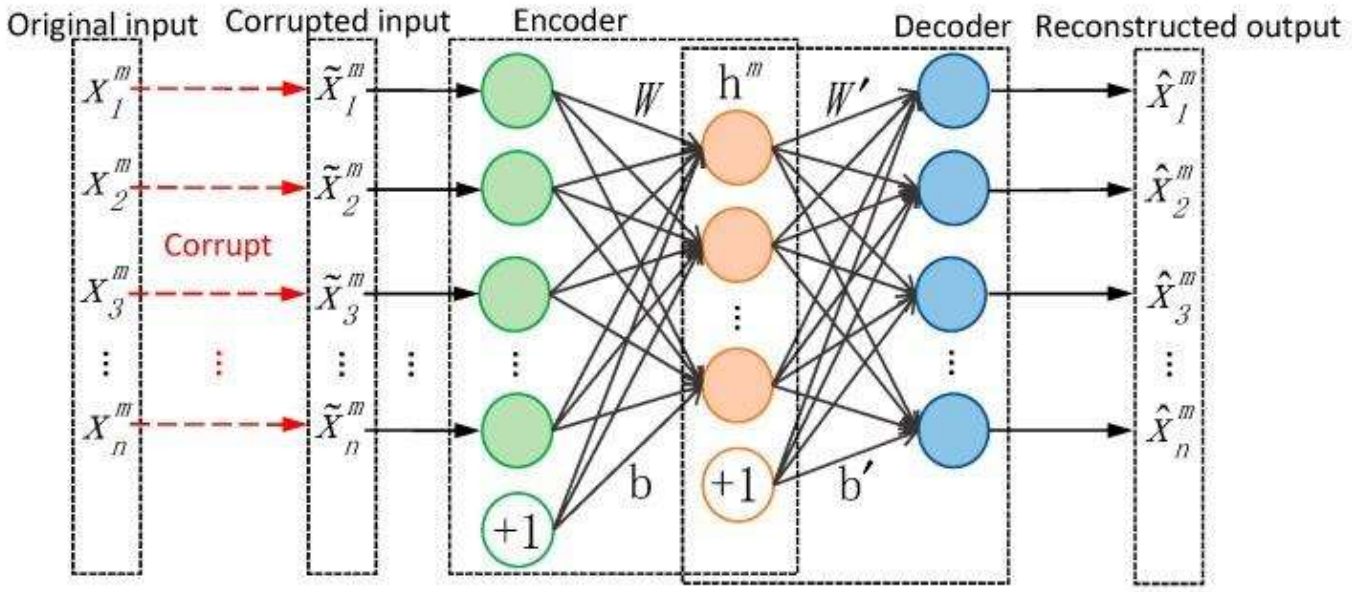
- Reconstruction MSE
- Classification Accuracy
- Precision

- Recall
- F1-score
- ROC-AUC

Conclusion

A denoising autoencoder is appropriate for noisy healthcare data because it learns to reconstruct clean patient information from corrupted inputs. This forces the latent representation to capture robust and useful features rather than memorizing noise.

Memory:
Add Noise → Learn Clean Representation → Reduce Overfitting



Q3. Design a Deep Generative Model

Problem

A recommendation/content-generation system needs to learn the complex probability distribution of its data.

A **Deep Belief Network (DBN)** can be used to learn hierarchical representations and generate new samples.

Proposed DBN Architecture

Input / Visible Layer

V

128 neurons

↓

Hidden Layer 1

64 neurons

↓

Hidden Layer 2

32 neurons

↓

Hidden Layer 3

16 neurons

↓

Learned Representation

↓

Generated / Reconstructed Data

A DBN is formed by stacking multiple probabilistic layers, commonly based on **Restricted Boltzmann Machines (RBMs)**.

1. Visible Layer

The visible layer receives the original input.

For a recommendation system, the input could represent:

User → Movie ratings/preferences

For content generation:

Image/Text Features

2. Hidden Layers

The hidden layers learn increasingly complex features.

Visible

↓

Hidden 1 → Low-level patterns

↓

Hidden 2 → Intermediate patterns

↓

Hidden 3 → High-level patterns

3. Learning Process

DBN training commonly uses **layer-wise unsupervised pre-training**.

Step 1

Train the first RBM:

Visible ↔ Hidden 1

Step 2

Use the learned hidden representation as input to the next RBM:

Hidden 1 ↔ Hidden 2

Step 3

Continue for deeper layers.

Step 4

Fine-tune the complete network for the final application.

4. Why DBN?

DBN is useful because it:

- Learns hierarchical representations.
- Models complex data distributions.
- Can work with limited labeled data.
- Performs unsupervised feature learning.
- Can be used for generation/reconstruction.

5. Evaluation

Depending on the application:

Recommendation:

- Precision
- Recall
- F1-score
- RMSE

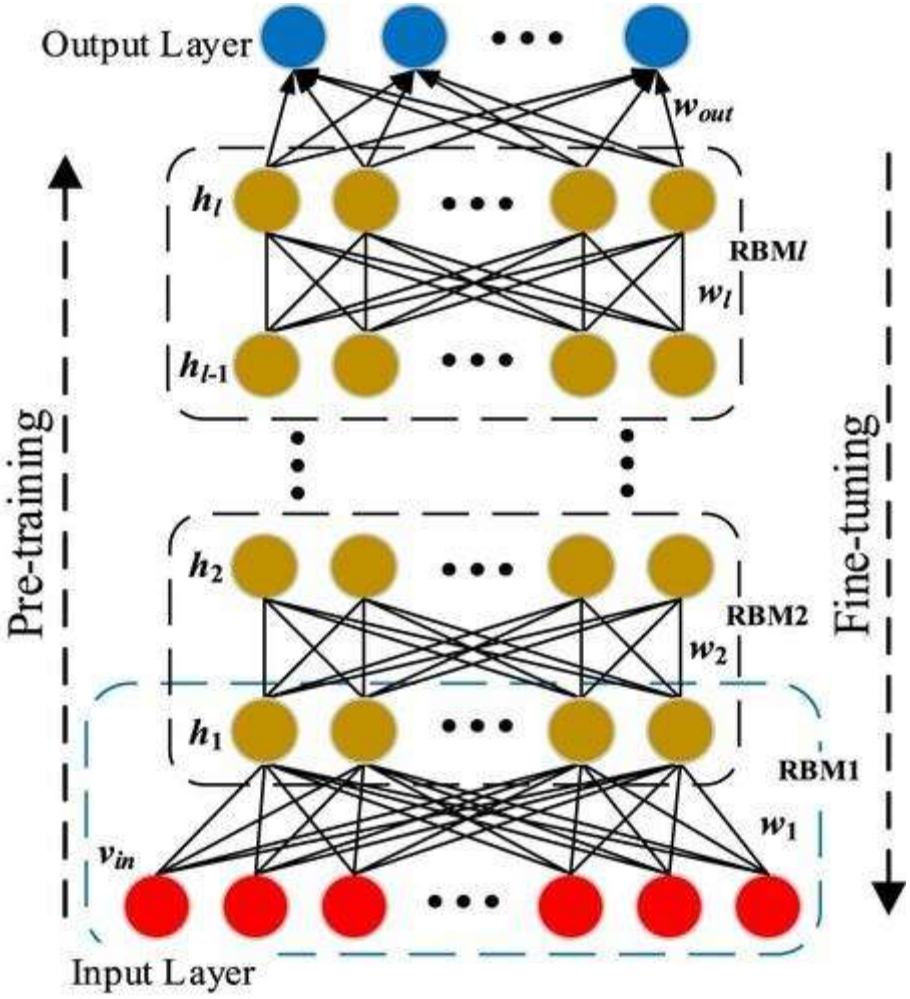
Generation:

- Reconstruction error
- Sample quality
- Diversity

Conclusion

A Deep Belief Network learns hierarchical representations through stacked probabilistic hidden layers. Layer-wise pre-training allows the model to learn useful features before fine-tuning, making it suitable for complex generative and recommendation applications.

Memory:
DBN = Stack RBMs → Learn features layer-by-layer → Fine-tune



Q4. Stochastic / Contractive Representation Model

Problem

An image or signal-processing system should produce stable representations even when the input has small variations.

A **Contractive Autoencoder (CAE)** is suitable.

Architecture

Input Image / Signal

↓

Preprocessing

↓

Encoder

↓

Latent Representation

↓

Decoder

↓

Reconstructed Input

Example:

Input: 784

↓

Dense: 256 + ReLU

↓

Dense: 64 + ReLU

↓

Latent: 16

↓

Dense: 64 + ReLU

↓

Dense: 256 + ReLU

↓

Output: 784

Contractive Encoder

The contractive autoencoder adds a penalty that encourages the encoder representation to change only slightly when the input changes slightly.

Objective Function

$$L = L_{reconstruction} + \lambda L_{contractive}$$

where:

$$L_{contractive} = \|J_f(x)\|_F^2$$

$J_f(x)$is the **Jacobian matrix of the encoder**.

What does the contractive penalty do?

Suppose:

Original Image

↓

Small rotation/noise

↓

Slightly Modified Image

The encoder should produce:

Representation 1 ≈ Representation 2

Therefore, the learned representation becomes more stable.

Robustness Mechanism

Small Input Change

↓

Contractive Encoder

↓

Small Representation Change

↓

Stable Features

Advantages

- Robust to small input variations.
- Learns stable representations.
- Reduces sensitivity to noise.
- Useful for image and signal processing.
- Improves downstream classification.

Evaluation

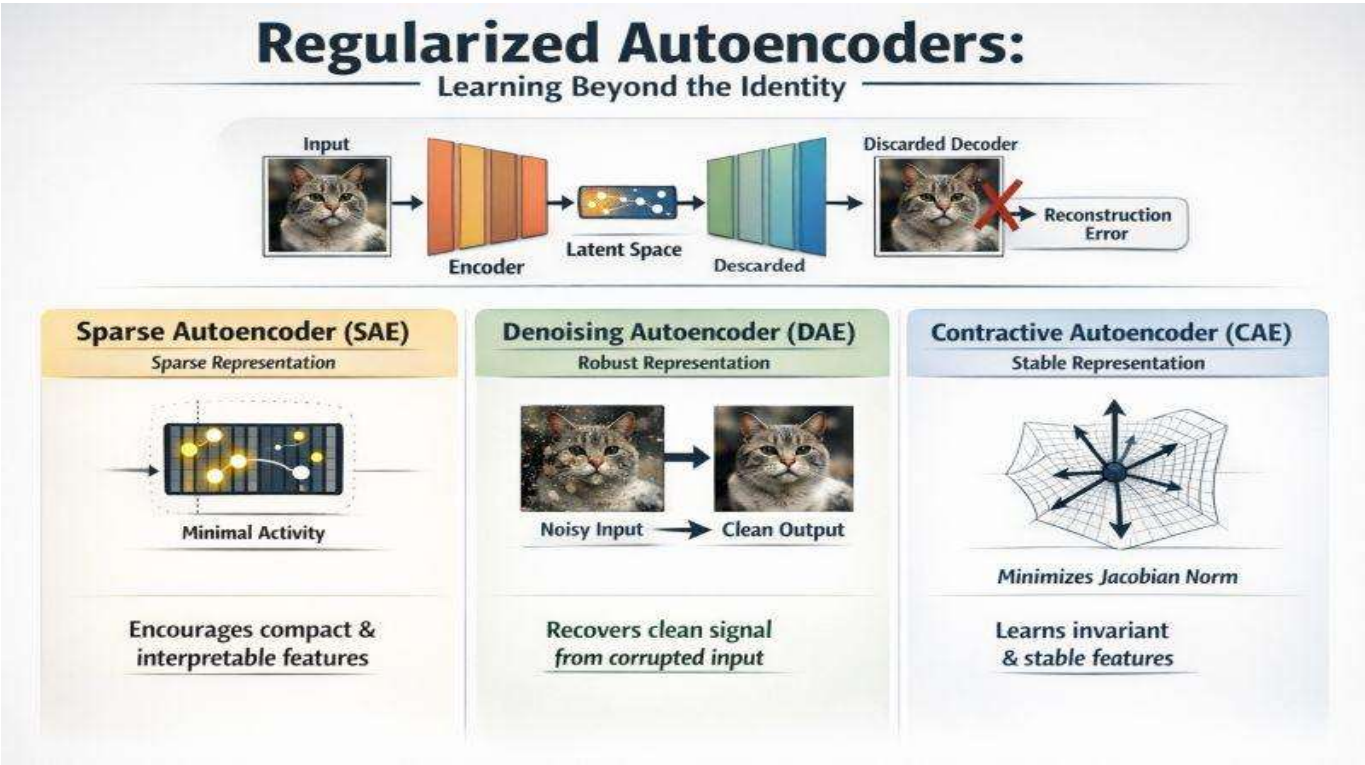
Use:

- Reconstruction MSE
- Classification accuracy
- Noise robustness
- Feature stability

Conclusion

A contractive autoencoder improves representation stability by penalizing excessive sensitivity of the encoder to small input changes. Therefore, similar inputs produce similar latent representations.

Memory:
Small Input Change → Small Feature Change = Stable Representation



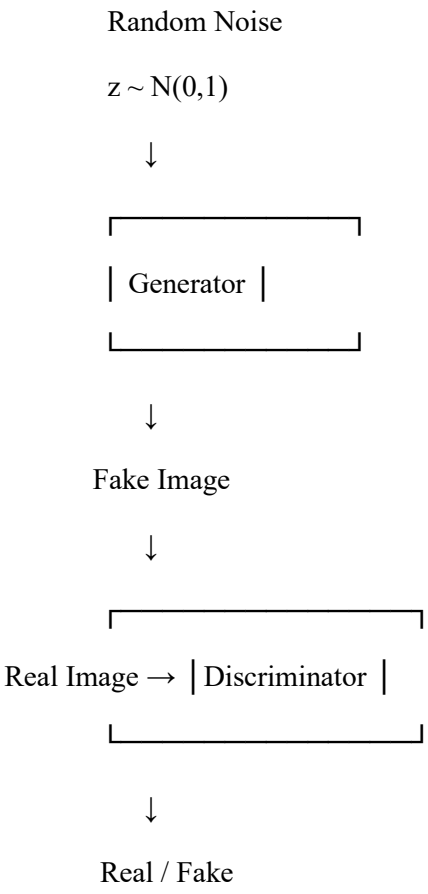
Q5. Design a GAN-Based Data Generation System

Problem

An organization has limited training images and wants to generate realistic synthetic images.

A **Generative Adversarial Network (GAN)** is suitable.

Architecture



1. Generator

The generator takes a random noise vector:

$$z \rightarrow G(z)$$

and produces a synthetic image.

Example:

Random Noise

↓

Dense Layer

↓

Upsampling / ConvTranspose

↓

Convolution

↓

Synthetic Image

The generator's objective is to create images that look real.

2. Discriminator

The discriminator receives:

- Real images
- Generated/fake images

and predicts whether the image is real or fake.

Image

↓

Convolution

↓

Feature Extraction

↓

Dense

↓

Sigmoid

↓

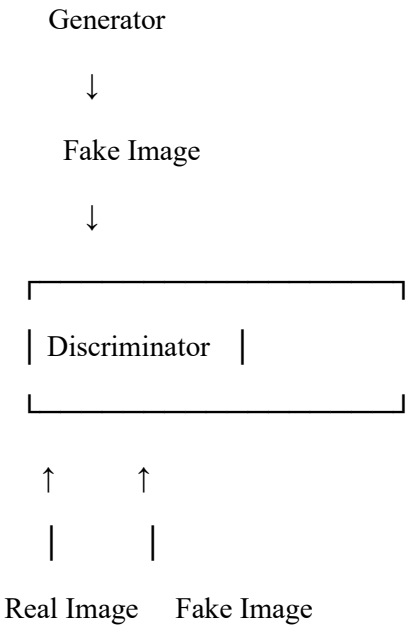
Real / Fake

3. Adversarial Training

The generator and discriminator compete against each other.

Random Noise

↓



Generator

Tries to **fool the discriminator**.

Discriminator

Tries to **correctly distinguish real and fake images**.

4. GAN Loss

The original GAN objective can be written as:

$$\min_G \max_D V(D,G) = E_{x\sim p_{data}}[\log D\,(x)] + E_{z\sim p_z}\left[\log\left(1 - D\big(G(z)\big)\right)\right]$$

In simple words:

The discriminator tries to correctly classify real and generated samples, while the generator tries to generate samples that the discriminator considers real.

5. Training Process

Step 1

Generate random noise.

Step 2

Generator creates a fake image.

Step 3

Give real and fake images to discriminator.

Step 4

Train discriminator.

Step 5

Train generator to fool discriminator.

Step 6

Repeat many times.

Eventually:

Random Noise

↓

Generator

↓

Realistic Synthetic Image

6. Evaluation

Generated samples can be evaluated using:

- Visual quality
- Diversity
- FID score
- Inception Score
- Downstream classification performance

7. GAN vs Autoencoder

Feature	**Autoencoder**	**GAN**
Main purpose	Reconstruction	Generation
Structure	Encoder + Decoder	Generator + Discriminator
Training	Reconstruction loss	Adversarial loss
Output	Reconstructed input	New synthetic sample

Feature	Autoencoder	GAN
Image quality	Can be blurry	Usually sharper/more realistic
Main strength	Feature learning	Realistic generation

Why GAN is suitable?

The organization needs **realistic new images**, not just compressed or reconstructed versions of existing images.

Therefore, GAN is more suitable.

Conclusion

A GAN can generate realistic synthetic training images using a generator and discriminator trained adversarially. It is particularly suitable when the organization has limited images and requires additional realistic samples for model training.

Memory:
Generator creates → Discriminator checks → Competition → Realistic samples